In turn, the value $\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})$ that solves this approximation is given by the solution to a classical dual norm problem ($|\cdot|^{q-1}$ denotes elementwise absolute value and power)[2]:

$$\hat{\boldsymbol{\epsilon}}(\boldsymbol{w}) = \rho \operatorname{sign}\left(\nabla_{\boldsymbol{w}} L_{\mathcal{S}}(\boldsymbol{w})\right) |\nabla_{\boldsymbol{w}} L_{\mathcal{S}}(\boldsymbol{w})|^{q-1} / \left(\|\nabla_{\boldsymbol{w}} L_{\mathcal{S}}(\boldsymbol{w})\|_q^q\right)^{1/p} \tag{2}$$

where $1/p + 1/q = 1$. Substituting back into equation (1) and differentiating, we then have

$$\begin{aligned}\nabla_{\boldsymbol{w}} L_{\mathcal{S}}^{SAM}(\boldsymbol{w}) \approx \nabla_{\boldsymbol{w}} L_{\mathcal{S}}(\boldsymbol{w} + \hat{\boldsymbol{\epsilon}}(\boldsymbol{w})) &= \frac{d(\boldsymbol{w} + \hat{\boldsymbol{\epsilon}}(\boldsymbol{w}))}{d\boldsymbol{w}} \nabla_{\boldsymbol{w}} L_{\mathcal{S}}(\boldsymbol{w})|_{\boldsymbol{w}+\hat{\boldsymbol{\epsilon}}(w)} \\ &= \nabla_{w} L_{\mathcal{S}}(\boldsymbol{w})|_{\boldsymbol{w}+\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})} + \frac{d\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})}{d\boldsymbol{w}} \nabla_{\boldsymbol{w}} L_{\mathcal{S}}(\boldsymbol{w})|_{\boldsymbol{w}+\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})}.\end{aligned}$$

This approximation to $\nabla_{\boldsymbol{w}} L_{\mathcal{S}}^{SAM}(\boldsymbol{w})$ can be straightforwardly computed via automatic differentiation, as implemented in common libraries such as JAX, TensorFlow, and PyTorch. Though this computation implicitly depends on the Hessian of $L_{\mathcal{S}}(\boldsymbol{w})$ because $\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})$ is itself a function of $\nabla_{\boldsymbol{w}} L_{\mathcal{S}}(\boldsymbol{w})$, the Hessian enters only via Hessian-vector products, which can be computed tractably without materializing the Hessian matrix. Nonetheless, to further accelerate the computation, we drop the second-order terms. obtaining our final gradient approximation:

$$\nabla_{\boldsymbol{w}} L_{\mathcal{S}}^{SAM}(\boldsymbol{w}) \approx \nabla_{\boldsymbol{w}} L_{\mathcal{S}}(w)|_{\boldsymbol{w}+\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})}. \tag{3}$$

As shown by the results in Section 3, this approximation (without the second-order terms) yields an effective algorithm. In Appendix C.4, we additionally investigate the effect of instead including the second-order terms; in that initial experiment, including them surprisingly degrades performance, and further investigating these terms' effect should be a priority in future work.

We obtain the final SAM algorithm by applying a standard numerical optimizer such as stochastic gradient descent (SGD) to the SAM objective $L_{\mathcal{S}}^{SAM}(\boldsymbol{w})$, using equation 3 to compute the requisite objective function gradients. Algorithm 1 gives pseudo-code for the full SAM algorithm, using SGD as the base optimizer, and Figure 2 schematically illustrates a single SAM parameter update.

Input: Training set $\mathcal{S} \triangleq \cup_{i=1}^{n}\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}$, Loss function $l : \mathcal{W} \times \mathcal{X} \times \mathcal{Y} \to \mathbb{R}_+$, Batch size b, Step size $\eta > 0$, Neighborhood size $\rho > 0$.
Output: Model trained with SAM
Initialize weights $\boldsymbol{w}_0$, $t = 0$;
while *not converged* **do**
  Sample batch $\mathcal{B} = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), ...(\boldsymbol{x}_b, \boldsymbol{y}_b)\}$;
  Compute gradient $\nabla_{\boldsymbol{w}} L_{\mathcal{B}}(\boldsymbol{w})$ of the batch's training loss;
  Compute $\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})$ per equation 2;
  Compute gradient approximation for the SAM objective (equation 3): $\boldsymbol{g} = \nabla_{w} L_{\mathcal{B}}(\boldsymbol{w})|_{\boldsymbol{w}+\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})}$;
  Update weights: $\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \boldsymbol{g}$;
  $t = t + 1$;
end
return $\boldsymbol{w}_t$

Algorithm 1: SAM algorithm

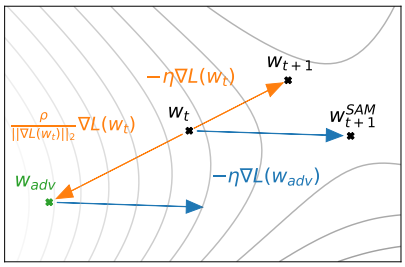

Figure 2: Schematic of the SAM parameter update.

3 Empirical Evaluation

In order to assess SAM's efficacy, we apply it to a range of different tasks, including image classification from scratch (including on CIFAR-10, CIFAR-100, and ImageNet), finetuning pretrained models, and learning with noisy labels. In all cases, we measure the benefit of using SAM by simply replacing the optimization procedure used to train existing models with SAM, and computing the resulting effect on model generalization. As seen below, SAM materially improves generalization performance in the vast majority of these cases.

[2] In the case of interest $p = 2$, this boils down to simply rescaling the gradient such that its norm is ρ.

3.1 Image Classification From Scratch

We first evaluate SAM's impact on generalization for today's state-of-the-art models on CIFAR-10 and CIFAR-100 (without pretraining): WideResNets with ShakeShake regularization (Zagoruyko & Komodakis, 2016; Gastaldi, 2017) and PyramidNet with ShakeDrop regularization (Han et al., 2016; Yamada et al., 2018). Note that some of these models have already been heavily tuned in prior work and include carefully chosen regularization schemes to prevent overfitting; therefore, significantly improving their generalization is quite non-trivial. We have ensured that our implementations' generalization performance in the absence of SAM matches or exceeds that reported in prior work (Cubuk et al., 2018; Lim et al., 2019)

All results use basic data augmentations (horizontal flip, padding by four pixels, and random crop). We also evaluate in the setting of more advanced data augmentation methods such as cutout regularization (Devries & Taylor, 2017) and AutoAugment (Cubuk et al., 2018), which are utilized by prior work to achieve state-of-the-art results.

SAM has a single hyperparameter ρ (the neighborhood size), which we tune via a grid search over $\{0.01, 0.02, 0.05, 0.1, 0.2, 0.5\}$ using 10% of the training set as a validation set[3]. Please see appendix C.1 for the values of all hyperparameters and additional training details. As each SAM weight update requires two backpropagation operations (one to compute $\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})$ and another to compute the final gradient), we allow each non-SAM training run to execute twice as many epochs as each SAM training run, and we report the best score achieved by each non-SAM training run across either the standard epoch count or the doubled epoch count[4]. We run five independent replicas of each experimental condition for which we report results (each with independent weight initialization and data shuffling), reporting the resulting mean error (or accuracy) on the test set, and the associated 95% confidence interval. Our implementations utilize JAX (Bradbury et al., 2018), and we train all models on a single host having 8 NVidia V100 GPUs[5]. To compute the SAM update when parallelizing across multiple accelerators, we divide each data batch evenly among the accelerators, independently compute the SAM gradient on each accelerator, and average the resulting sub-batch SAM gradients to obtain the final SAM update.

As seen in Table 1, SAM improves generalization across all settings evaluated for CIFAR-10 and CIFAR-100. For example, SAM enables a simple WideResNet to attain 1.6% test error, versus 2.2% error without SAM. Such gains have previously been attainable only by using more complex model architectures (e.g., PyramidNet) and regularization schemes (e.g., Shake-Shake, ShakeDrop); SAM provides an easily-implemented, model-independent alternative. Furthermore, SAM delivers improvements even when applied atop complex architectures that already use sophisticated regularization: for instance, applying SAM to a PyramidNet with ShakeDrop regularization yields 10.3% error on CIFAR-100, which is, to our knowledge, a new state-of-the-art on this dataset without the use of additional data.

Beyond CIFAR-{10, 100}, we have also evaluated SAM on the SVHN (Netzer et al., 2011) and Fashion-MNIST datasets (Xiao et al., 2017). Once again, SAM enables a simple WideResNet to achieve accuracy at or above the state-of-the-art for these datasets: 0.99% error for SVHN, and 3.59% for Fashion-MNIST. Details are available in appendix B.1.

To assess SAM's performance at larger scale, we apply it to ResNets (He et al., 2015) of different depths (50, 101, 152) trained on ImageNet (Deng et al., 2009). In this setting, following prior work (He et al., 2015; Szegedy et al., 2015), we resize and crop images to 224-pixel resolution, normalize them, and use batch size 4096, initial learning rate 1.0, cosine learning rate schedule, SGD optimizer with momentum 0.9, label smoothing of 0.1, and weight decay 0.0001. When applying SAM, we use $\rho = 0.05$ (determined via a grid search on ResNet-50 trained for 100 epochs). We train all models on ImageNet for up to 400 epochs using a Google Cloud TPUv3 and report top-1 and top-5 test error rates for each experimental condition (mean and 95% confidence interval across 5 independent runs).

[3] We found $\rho = 0.05$ to be a solid default value, and we report in appendix C.3 the scores for all our experiments, obtained with $\rho = 0.05$ without further tuning.

[4] Training for longer generally did not improve accuracy significantly, except for the models previously trained for only 200 epochs and for the largest, most regularized model (PyramidNet + ShakeDrop).

[5] Because SAM's performance is amplified by not syncing the perturbations, data parallelism is highly recommended to leverage SAM's full potential (see Section 4 for more details).

		CIFAR-10		CIFAR-100	
Model	Augmentation	SAM	SGD	SAM	SGD
WRN-28-10 (200 epochs)	Basic	**2.7**$_{\pm 0.1}$	3.5$_{\pm 0.1}$	**16.5**$_{\pm 0.2}$	18.8$_{\pm 0.2}$
WRN-28-10 (200 epochs)	Cutout	**2.3**$_{\pm 0.1}$	2.6$_{\pm 0.1}$	**14.9**$_{\pm 0.2}$	16.9$_{\pm 0.1}$
WRN-28-10 (200 epochs)	AA	**2.1**$_{\pm <0.1}$	2.3$_{\pm 0.1}$	**13.6**$_{\pm 0.2}$	15.8$_{\pm 0.2}$
WRN-28-10 (1800 epochs)	Basic	**2.4**$_{\pm 0.1}$	3.5$_{\pm 0.1}$	**16.3**$_{\pm 0.2}$	19.1$_{\pm 0.1}$
WRN-28-10 (1800 epochs)	Cutout	**2.1**$_{\pm 0.1}$	2.7$_{\pm 0.1}$	**14.0**$_{\pm 0.1}$	17.4$_{\pm 0.1}$
WRN-28-10 (1800 epochs)	AA	**1.6**$_{\pm 0.1}$	2.2$_{\pm <0.1}$	**12.8**$_{\pm 0.2}$	16.1$_{\pm 0.2}$
Shake-Shake (26 2x96d)	Basic	**2.3**$_{\pm <0.1}$	2.7$_{\pm 0.1}$	**15.1**$_{\pm 0.1}$	17.0$_{\pm 0.1}$
Shake-Shake (26 2x96d)	Cutout	**2.0**$_{\pm <0.1}$	2.3$_{\pm 0.1}$	**14.2**$_{\pm 0.2}$	15.7$_{\pm 0.2}$
Shake-Shake (26 2x96d)	AA	**1.6**$_{\pm <0.1}$	1.9$_{\pm 0.1}$	**12.8**$_{\pm 0.1}$	14.1$_{\pm 0.2}$
PyramidNet	Basic	**2.7**$_{\pm 0.1}$	4.0$_{\pm 0.1}$	**14.6**$_{\pm 0.4}$	19.7$_{\pm 0.3}$
PyramidNet	Cutout	**1.9**$_{\pm 0.1}$	2.5$_{\pm 0.1}$	**12.6**$_{\pm 0.2}$	16.4$_{\pm 0.1}$
PyramidNet	AA	**1.6**$_{\pm 0.1}$	1.9$_{\pm 0.1}$	**11.6**$_{\pm 0.1}$	14.6$_{\pm 0.1}$
PyramidNet+ShakeDrop	Basic	**2.1**$_{\pm 0.1}$	2.5$_{\pm 0.1}$	**13.3**$_{\pm 0.2}$	14.5$_{\pm 0.1}$
PyramidNet+ShakeDrop	Cutout	**1.6**$_{\pm <0.1}$	1.9$_{\pm 0.1}$	**11.3**$_{\pm 0.1}$	11.8$_{\pm 0.2}$
PyramidNet+ShakeDrop	AA	**1.4**$_{\pm <0.1}$	1.6$_{\pm <0.1}$	**10.3**$_{\pm 0.1}$	10.6$_{\pm 0.1}$

Table 1: Results for SAM on state-of-the-art models on CIFAR-{10, 100} (WRN = WideResNet; AA = AutoAugment; SGD is the standard non-SAM procedure used to train these models).

As seen in Table 2, SAM again consistently improves performance, for example improving the ImageNet top-1 error rate of ResNet-152 from 20.3% to 18.4%. Furthermore, note that SAM enables increasing the number of training epochs while continuing to improve accuracy without overfitting. In contrast, the standard training procedure (without SAM) generally significantly overfits as training extends from 200 to 400 epochs.

Model	Epoch	SAM		Standard Training (No SAM)	
		Top-1	Top-5	Top-1	Top-5
ResNet-50	100	**22.5**$_{\pm 0.1}$	6.28$_{\pm 0.08}$	22.9$_{\pm 0.1}$	6.62$_{\pm 0.11}$
	200	**21.4**$_{\pm 0.1}$	5.82$_{\pm 0.03}$	22.3$_{\pm 0.1}$	6.37$_{\pm 0.04}$
	400	**20.9**$_{\pm 0.1}$	5.51$_{\pm 0.03}$	22.3$_{\pm 0.1}$	6.40$_{\pm 0.06}$
ResNet-101	100	**20.2**$_{\pm 0.1}$	5.12$_{\pm 0.03}$	21.2$_{\pm 0.1}$	5.66$_{\pm 0.05}$
	200	**19.4**$_{\pm 0.1}$	4.76$_{\pm 0.03}$	20.9$_{\pm 0.1}$	5.66$_{\pm 0.04}$
	400	**19.0**$_{\pm <0.01}$	4.65$_{\pm 0.05}$	22.3$_{\pm 0.1}$	6.41$_{\pm 0.06}$
ResNet-152	100	**19.2**$_{\pm <0.01}$	4.69$_{\pm 0.04}$	20.4$_{\pm <0.0}$	5.39$_{\pm 0.06}$
	200	**18.5**$_{\pm 0.1}$	4.37$_{\pm 0.03}$	20.3$_{\pm 0.2}$	5.39$_{\pm 0.07}$
	400	**18.4**$_{\pm <0.01}$	4.35$_{\pm 0.04}$	20.9$_{\pm <0.0}$	5.84$_{\pm 0.07}$

Table 2: Test error rates for ResNets trained on ImageNet, with and without SAM.

3.2 Finetuning

Transfer learning by pretraining a model on a large related dataset and then finetuning on a smaller target dataset of interest has emerged as a powerful and widely used technique for producing high-quality models for a variety of different tasks. We show here that SAM once again offers considerable benefits in this setting, even when finetuning extremely large, state-of-the-art, already high-performing models.

In particular, we apply SAM to finetuning EfficentNet-b7 (pretrained on ImageNet) and EfficientNet-L2 (pretrained on ImageNet plus unlabeled JFT; input resolution 475) (Tan & Le, 2019; Kornblith et al., 2018; Huang et al., 2018). We initialize these models to publicly available checkpoints[6] trained with RandAugment (84.7% accuracy on ImageNet) and NoisyStudent (88.2% accuracy on ImageNet), respectively. We finetune these models on each of several target datasets by training each model starting from the aforementioned checkpoint; please see the appendix for details of the hyperparameters used. We report the mean and 95% confidence interval of top-1 test error over 5 independent runs for each dataset.

[6] https://github.com/tensorflow/tpu/tree/master/models/official/efficientnet